COS40007 – Artificial Intelligence Engineering

Applied Project Task 1 (Group Task 1)

System Blueprint & Project Planning

Project: Anti-Spoofing for Secure Login – Edge Liveness Detection on Raspberry Pi 5

Unit	COS40007 Artificial Intelligence Engineering
Task	Group Task 1 – System Blueprint & Project Planning (Studio Activity 5, Week 4)
Group No.	3
Group members	Dinh Phu Nguyen – Student ID: 105027282 Phuc Lam Nguyen – Student ID: 105006540
Submission date	25/10/2026
GitHub repository	https://github.com/phu1009/COS40007---AI-Engineering-

Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, the Traditional Owners of the land on which Swinburne University of Technology's Australian campuses stand, and we pay our respects to their Elders past, present and emerging. We also acknowledge Aboriginal and Torres Strait Islander peoples as the Traditional Custodians of lands across Australia, and recognise their continuing connection to land, waters, culture and community.

1. Group Agreement and Project Plan

- Studio Session No.: 5 (Week 4 Studio Activities – Group Task 1)
- Group No. (given by tutor only): 3
- Name and Students' Numbers of Group Members: Dinh Phu Nguyen (105027282); Phuc Lam Nguyen (105006540)

Group style and roles

Who does what, and how will you work together? Who will be responsible for the submitted deliverables? Who will ask questions on behalf of their group? Who is the Scrum Master for each Sprint or all sprints?

We are a two-member team. M1 leads the project and holds most technical roles; M2 supports with research and data preparation. We follow a light-weight Scrum process: one 1-week planning sprint and four 2-week delivery sprints, a sprint backlog in Jira, and a short review and retrospective at the end of every sprint. Every deliverable has one **owner** and one **reviewer**, so nothing is submitted without a second check, and every Jira task names a **backup owner** (the other member) in case of illness or non-delivery.

Member	Roles	Main responsibilities	Owns these deliverables
M1 – Dinh Phu Nguyen	Scrum Master (all sprints) Group contact with tutor ML Engineer Edge / embedded engineer (Raspberry Pi) MLOps & DevOps lead Data, web/backend, QA & documentation	• Runs sprint planning, review and retrospective; keeps the sprint on schedule • Asks questions on behalf of the group and uploads all submissions to Canvas • Model training, comparison, threshold selection, ONNX / INT8 export • Raspberry Pi pipeline: camera capture, on-device preprocessing, inference, decision engine • GitHub repository rules, CI/CD workflows, model versioning • Data pipeline, web login UI, backend API, automated tests and documentation	All Task 1 sections Data pipeline & model package Pi edge service Web login prototype Reports and final submission
M2 – Phuc Lam Nguyen	Research assistant Data support	• Background research on candidate datasets and related work • Small data-preparation tasks (e.g. helping collect the consented Pi-camera test set)	Dataset research notes
Both	Reviewers of each other's work	• Weekly meeting, peer review of every pull request and report section, demo rehearsal, final report	Group report and demo video

Decision rights. The role owner makes technical decisions inside their role and records them as a design decision (Section 4.4). Scope changes, and anything that affects marks or deadlines, are decided together in the weekly meeting. The tutor acts as the project stakeholder; M1 acts as product-owner proxy and orders the backlog, which M2 confirms at sprint planning.

Timeline (Follow Sprint here)

By when do you need to make decisions? When do tasks need to be completed by to ensure you submit on time? (Aim to submit BEFORE the deadline to allow for unforeseen circumstances).

Week numbers follow the teaching calendar on Canvas; the exact dates are set as sprint dates in Jira. Our **internal deadline is 48 hours before every official deadline**, each sprint keeps about 20% spare capacity for unplanned work, and Week 12 contains no new features.

Sprint	Weeks	Sprint goal	Deliverables (definition of done)	Lead	Decision point / internal deadline
S0	Wk 4	Plan the project and set up tools	Group agreement, sprint plan, system blueprint, MLOps/DevOps design, Jira board, GitHub repository, documentation page → Group Task 1 PDF	M1	Scope fixed (passive liveness first) and datasets chosen by end of Wk 4
S1	Wk 5–6	Data pipeline ready; Pi capturing frames	CelebA-Spoof downloaded; Replay-Attack EULA requested through tutor; manifest + DVC; EDA; subject-disjoint split with frozen test set; shared preprocessing module; Pi 5 set up with camera capture and face-detection benchmark	M1 (M2: dataset research)	Test split frozen by end of Wk 5; Pi capture working by end of Wk 6
S2	Wk 7–8	Train, compare and select the model	LBP+SVM and pre-trained MiniFASNet baselines; fine-tuned MiniFASNet and MobileNetV3-Small; MLflow runs; evaluation script; threshold on validation; ONNX + INT8 export; Pi latency benchmark	M1	Model selected by end of Wk 8 (APCER/BPCER on validation + Pi p95 latency)
S3	Wk 9–10	Integrate the end-to-end login flow	Pi edge service with decision engine and signed results; backend auth API; web login UI with all guidance states; logging without images; GitHub Actions CI (lint + tests)	M1	Feature freeze at end of Wk 10
S4	Wk 11–12	Evaluate, monitor, document and demo	One-time test-set evaluation per attack type; Pi-camera sanity test (consented); monitoring and drift report; full automated test suite; final report and demo video; retrospective	M1 (M2: Pi-camera data collection)	Final report complete 3 days before deadline; demo rehearsal in Wk 12

Simple timeline (Gantt view). Coloured cells show when each workstream is active and who leads it; colours match the sprint badges in the system blueprint (Figure 1).

Workstream	Wk 4 (S0)	Wk 5 (S1)	Wk 6 (S1)	Wk 7 (S2)	Wk 8 (S2)	Wk 9 (S3)	Wk 10 (S3)	Wk 11 (S4)	Wk 12 (S4)
Planning, Jira & GitHub setup	M1								
Dataset research	M2	M2							
Data pipeline, EDA & split		M1	M1	M1					
Raspberry Pi edge pipeline		M1	M1		M1	M1	M1		

Workstream	Wk 4 (S0)	Wk 5 (S1)	Wk 6 (S1)	Wk 7 (S2)	Wk 8 (S2)	Wk 9 (S3)	Wk 10 (S3)	Wk 11 (S4)	Wk 12 (S4)
Model training & selection				M1	M1				
Web UI & backend integration						M1	M1		
Testing & evaluation					M1		M1	M1	M1
MLOps: CI/CD, monitoring, drift	M1						M1	M1	M1
Report, documentation & demo	M1							M1	M1

Scope control (MoSCoW). *Must*: passive liveness running on the Pi, web login demo, APCER/BPCER/ACER and latency report, fail-safe states. *Should*: optional blink challenge as a second step, drift dashboard. *Could*: Camera Module 3 NoIR with infrared lighting, comparison with a Hailo AI HAT+. *Won't (this semester)*: face recognition / identity matching, a native mobile app, 3D-mask attacks. Keeping the Must list small is what makes this plan realistic for a two-person team.

Communication Plan

Decide and document how your group will communicate and collaborate regularly throughout the project. This could include meeting via Zoom, communicating via a Teams group or Canvas, and using cloud services for shared documents, etc. How often and how will you communicate? What are your expectations of others responding to your communications?

Channel	Frequency / timing	Purpose and rules
Sprint meeting (Microsoft Teams or in person after the studio)	Weekly, Monday, 45 min	Progress, blockers, next tasks. Minutes are written in the documentation page within 24 h and actions are added to Jira.
Asynchronous stand-up (group chat on Teams/Messenger)	Mon, Wed, Fri by 21:00	Three lines: done / next / blocked. Missing two stand-ups in a row starts the tension process below.
Sprint review + retrospective	Last day of each sprint, 30 min	Demo of finished work against the definition of done; one thing to keep, one to change.
Jira	Continuously	Single source of truth for tasks, owners, priorities and sprint progress.
GitHub	Continuously	Code, pull-request reviews and issues; branch names and commits carry the Jira key (e.g. SCRUM-23).
Google Drive / OneDrive + Notion (or Word)	Continuously	Report drafts, meeting minutes, design decisions and documentation.
Canvas / email to tutor	As needed	Official questions (asked by M1 on behalf of the group) and all submissions.

Response expectations. Messages are answered within 24 hours on weekdays and 48 hours at weekends; within 6 hours when a deadline is less than 48 hours away. Pull requests are reviewed within 48 hours. A member who will be unavailable posts a notice in advance and hands over open tasks.

On-track behaviours
What behaviours will result in great group outcomes?

- Updating Jira when starting or finishing a task, and linking commits and pull requests to Jira keys so work is visible.
- Meeting the internal deadline (48 h early), or warning the team at least 48 h ahead when a task is at risk.
- Small, reviewed pull requests with tests; no direct pushes to the main branch.
- Asking for help within one day of being blocked instead of waiting until the deadline.
- Reporting results honestly, including poor ones, and recording contributions and GenAI use accurately.
- Respecting dataset licences and privacy: no dataset files, face images or credentials shared outside the team's approved storage.

Off-track behaviours
What behaviours would result in group outcomes that are not great?

- Missing meetings without notice, or not responding for more than 48 hours.
- Missing internal deadlines without warning, or submitting unreviewed, last-minute work.
- Working outside Jira and GitHub so that work cannot be seen, reviewed or verified.
- Copying code or text without attribution, or using GenAI without declaring it.
- Tuning the model on the test set or reporting only favourable metrics.
- Storing or sharing raw face images, credentials or dataset files outside approved storage.

Resolving tensions
How will we resolve tensions or off-track behaviour as a group?

We use a four-step escalation path so problems are dealt with early and fairly:

- **1. Talk early.** Raise the issue privately and respectfully within 24 hours, focusing on the task rather than the person, and agree on a fix and a new date.
- **2. Record and re-plan.** If it is not resolved, discuss it at the weekly meeting; the agreed actions, new owner and date are written in the minutes and in Jira.

- **3. Formal notice and reallocation.** If a member misses two agreed commitments in one sprint or is unresponsive for 72 hours, the other member sends a written notice in the group chat. The affected tasks move to the backup owner and the change is logged in Jira, so contribution records stay accurate.
- **4. Escalate to the tutor.** If the problem continues into the next sprint, we contact the tutor before the next submission with evidence (Jira history, GitHub commits, meeting minutes) and ask for mediation. Contribution statements always reflect the work actually done.

Technical disagreements are settled with evidence (validation metrics, latency measured on the Pi) and time-boxed to 30 minutes; if there is still no agreement, the role owner decides and records the reasoning as a design decision.

Team Expectations:

1.	**Commitment and reliability:** attend the weekly meeting, finish sprint tasks by the internal deadline (48 h before the official deadline) and keep Jira up to date.
2.	**Quality and review:** every deliverable (code, model, report section) is reviewed by the other member before submission; code is merged only through a pull request with passing checks.
3.	**Integrity and privacy:** contributions, results and GenAI use are reported honestly; dataset licences are respected and no raw face images, credentials or dataset files leave approved storage.

2. **System Blueprint Diagram: A diagram showing system components, data flow, and ML model integration with an explanation relevant to your applied project.**

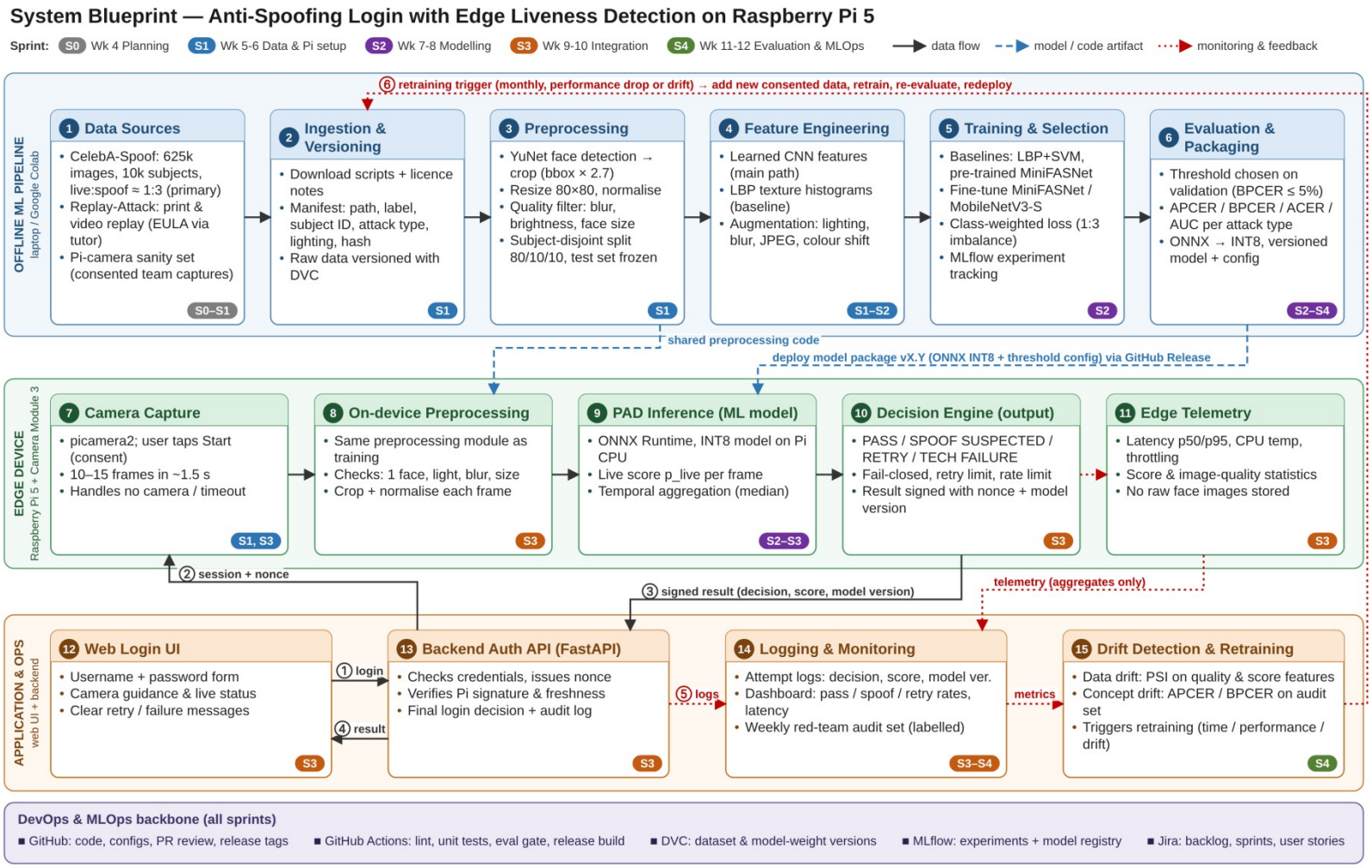


Figure 1. System blueprint: components, data flow, ML model integration and the sprint in which each stage is built. Editable draw.io source: Group Project/docs/architecture/system_blueprint.drawio in the group GitHub repository.

2.1 How the system works

The system has three layers connected by a shared DevOps/MLOps backbone. The **offline ML pipeline** (stages 1–6) turns public presentation-attack datasets into a versioned, validated model package. The **Raspberry Pi edge device** (stages 7–11) runs the liveness check next to the camera, so face images never leave the device; it outputs a decision with a confidence score. The **application and operations layer** (stages 12–15) combines that decision with the password check in the web login flow and watches the system in production, closing the retraining loop.

Login sequence (numbered arrows in Figure 1). ① The user submits username and password in the web UI. ② The backend checks the password and sends a short-lived session ID and random nonce to the Pi. ③ The Pi captures frames, runs the model and returns a result signed with its device key and bound to that nonce. ④ The backend verifies the signature and freshness, applies rate limits and returns the final decision to the UI. ⑤ Every attempt is logged without images. ⑥ Monitoring can trigger retraining, which feeds back into stage 2.

Stage (Figure 1)	What happens in our project	Why – design rationale and dataset (EDA) insight	Sprint
Data sources & ingestion (1–2)	CelebA-Spoof is the main dataset (625,537 images, 10,177 subjects; print, replay and paper-mask attacks; 4 lighting conditions). Replay-Attack (print and phone/tablet replay, 320×240 webcam) is used for cross-dataset testing; a small consented Pi-camera set for testing only. Scripts build a DVC-versioned manifest (path, label, subject, attack type, lighting, hash).	Licences forbid redistribution, so raw data never goes to GitHub; the manifest makes every split reproducible. Replay-Attack needs a staff EULA, so we request it through the tutor.	S0–S1
Preprocessing (3)	YuNet face detection; crop the box enlarged 2.7×; resize to 80×80; normalise; quality filter (blur, brightness, face size); subject-disjoint 80/10/10 split; near-duplicate check; test set frozen before any tuning.	Each subject has many images, so a random split would leak identities into the test set. The same module is reused on the Pi to avoid training/serving skew.	S1
Feature engineering (4)	The CNN learns features end-to-end; LBP texture histograms feed the classical baseline; augmentation varies lighting, blur, JPEG quality and colour.	CelebA-Spoof labels four lighting conditions (normal, strong, back-lit, dark), and the Pi camera differs from the dataset cameras, so augmentation targets these shifts.	S1–S2
ML model (5–6)	Compare LBP+SVM, pre-trained MiniFASNet, fine-tuned MiniFASNet and MobileNetV3-Small (class-weighted loss). Threshold set on validation; APCER, BPCER, ACER, precision, recall, F1 and ROC-AUC per attack type. Export ONNX → INT8.	Live:spoof ≈ 1:3, so accuracy misleads; we pick the lowest APCER at BPCER ≤ 5% within the Pi latency budget (ISO/IEC 30107-3 metrics). Models under 5 MB run on the Pi CPU.	S2 (final test S4)
Edge inference & model output (7–11)	After the user taps Start, the Pi captures 10–15 frames, preprocesses them, runs ONNX Runtime and takes the median live score. Output: PASS, SPOOF SUSPECTED, RETRY (low quality) or TECHNICAL FAILURE, with score, threshold, model version and processing time.	Biometric images stay on the device; multi-frame aggregation reduces single-frame errors; technical errors fail closed (spec requirement).	S1, S2–S3
UI & system integration (12–13)	Web login page with camera guidance and clear status messages; FastAPI backend that issues the nonce, verifies the Pi signature, enforces retry and rate limits and makes the final decision.	Login continues only when the password and the liveness check both pass; signing stops forged or replayed "pass" messages.	S3
Monitoring & retraining (14–15)	Structured attempt logs (no images), dashboard, weekly red-team audit with known attacks, drift detection that triggers retraining.	Explained in Section 3.	S3–S4

3. MLOps & DevOps Integration (High-Level Design): How MLOps (retraining, monitoring) and DevOps (CI/CD, version control) will fit into your project.

Our MLOps goal is a model that can be **reproduced**, **deployed safely to the Raspberry Pi** and **kept reliable** as lighting, cameras and attack methods change. One project constraint shapes the whole design: the specification forbids storing raw face images by default, so we cannot collect production images for retraining. New training data therefore comes only from public datasets and explicitly consented collection sessions, and production monitoring works on aggregate statistics.

3.1 Data & training strategy

- **Split.** CelebA-Spoof is split 80/10/10 into training, validation and test sets by **subject**, stratified by live/spoof and attack type. Replay-Attack keeps its official train/devel/test protocol (no client overlap) and is used as a cross-dataset test. The consented Pi-camera set is test-only.
- **Frozen test set.** The test set is frozen (hash stored in the manifest) before tuning. Model, threshold and hyper-parameters are chosen on validation data only; the test set is run once per release candidate, for reporting, never for selection.
- **Leakage controls.** Automated checks for subject overlap and near-duplicate frames (perceptual hash) run in CI; augmentation is applied to training data only.
- **New data for retraining – yes, under control.** New data comes from (a) new public dataset versions, (b) consented Pi-camera sessions recorded by the team (live and attack samples), and (c) red-team sessions with new attack instruments such as a new phone or OLED screen. It enters through the same ingestion pipeline as a new DVC version; new subjects go to training/validation, while a fixed Pi-domain test set is kept so results stay comparable.

3.2 Retraining strategy

Trigger	Rule (our thresholds)	Action
Time-based	Monthly retraining review (during the semester: at the end of each sprint).	Retrain if new data is ≥ 10% of the training set or a new attack type was collected; otherwise record "no change" in the model log.
Performance-based	Weekly red-team audit shows APCER > 10% or BPCER > 10%, or ACER worse than the released model by more than 3 percentage points.	Collect labelled examples of the failure cases, retrain, re-evaluate.
Drift-based	PSI > 0.2 on input-quality features or on the live-score distribution for two consecutive weeks (Section 3.3).	Investigate the cause; if the audit set confirms a drop, retrain with data covering the new conditions.
Event-based	New camera or hardware, a newly discovered attack instrument, or an ONNX Runtime / OS upgrade.	Re-run the evaluation and Pi benchmark; retrain if the gates in the next paragraph fail.

Frequency and process. We expect about one retraining per month (at most one per sprint during the semester), plus ad-hoc runs when a trigger fires. Retraining follows a **champion–challenger** process: the new model must (1) match or beat the current model's APCER at BPCER ≤ 5% on validation and on the red-team audit set,

(2) keep per-frame inference p95 ≤ 50 ms and a full attempt p95 ≤ 2 s on the Pi 5, and (3) pass all automated tests. It is then registered, released, run in **shadow mode** on the Pi (scores logged, decisions not used) for one audit session, and only then promoted. The previous version stays available for one-step rollback.

3.3 Monitoring & drift detection

What we monitor	Metrics	Where / how	Alert when
Service health (Pi + backend)	Latency p50/p95 per stage, CPU temperature and throttling flags, memory, technical-failure rate	Pi telemetry → backend log store → dashboard	p95 attempt > 2 s or failure rate > 1%
Model behaviour (no labels)	Distribution of live scores, pass / spoof / retry rates, share of scores close to the threshold	Structured attempt logs	Pass rate changes by > 15% vs. the reference week
Input data quality	Brightness, blur (Laplacian variance), face size, head pose, number of faces	Computed on the Pi, sent as aggregates	PSI > 0.2
Labelled performance	APCER, BPCER, ACER per attack type	Weekly red-team audit: known live and attack sessions	APCER or BPCER > 10%

Data drift means the inputs change, for example new room lighting or a different camera. We compare each week's distributions of the input-quality features and live scores with a reference window (validation statistics and the first deployment week) using the Population Stability Index and, for continuous features, the Kolmogorov–Smirnov test.

Concept drift means the relationship between input and label changes, for example a new high-resolution OLED phone or better prints that the model now accepts as live. Input statistics can look normal while this happens, so we detect it through the labelled weekly red-team audit (rising APCER), a growing share of borderline scores, and reported false accepts or rejects. The response is to collect labelled samples of the new attack, retrain, and update the list of attacks documented as out of scope. All monitoring uses scores and aggregate statistics only; no face images are logged.

3.4 DevOps practices

Version control strategy. We will use GitHub. All code, configuration and documentation live in one group repository. The main branch is protected: changes arrive only through pull requests with one review and passing checks. Short-lived feature branches are named after Jira keys (e.g. feature/SCRUM-23-pi-capture) and commit messages start with the key, so Jira and GitHub stay linked. Releases are tagged vMAJOR.MINOR.PATCH. Datasets, face images and secrets are excluded by .gitignore; secrets live in GitHub Secrets.

CI/CD pipeline concept (GitHub Actions – to be implemented in a later task).

Workflow	Trigger	Steps
CI – ci.yml	Every pull request and push to main	Lint and format check (ruff); unit tests with pytest, using recorded videos instead of a real camera; manifest schema and leakage checks; smoke inference on 20 fixture images comparing FP32 and INT8 outputs.
Train & evaluate – train.yml	Manual dispatch or monthly schedule	Run training on a GPU runner (Colab or a self-hosted runner, because GitHub-hosted runners have no GPU); log to MLflow; apply the evaluation gate from Section 3.2; fail the workflow if the gate is not met.

Workflow	Trigger	Steps
Release / CD – release.yml	Git tag v*	Build the model package (ONNX INT8 + threshold config + model card + checksum) and publish a GitHub Release. The Pi pulls the release (updater service or self-hosted runner), verifies the checksum, runs an on-device smoke test and latency check, starts shadow mode, then promotes; it rolls back automatically if the smoke test fails.

Code and model versioning.

Artefact	Tool	Version identifier
Source code, configs (threshold, preprocessing)	Git + GitHub	Commit SHA and release tag
Datasets, splits, manifests	DVC (remote: shared Google Drive)	DVC hash + dataset version, e.g. data-v1.2
Experiments	MLflow tracking	Run ID with parameters, metrics, Git SHA and data version
Trained models	MLflow Model Registry + GitHub Releases	pad-model vMAJOR.MINOR.PATCH (major = new architecture, minor = retrain, patch = threshold/config change)
Runtime environment	Pinned requirements file + Pi setup notes	Locked package versions per release

Every login attempt record stores the model version and config version, so any decision can be traced back to the exact code commit, dataset version and training run.

4. Project Management Setup (Agile Tools): A Jira (or Trello or others) board with key user stories and a Confluence/Notion/Word page for project documentation.

4.1 Jira board

We use **Jira Software (Scrum template)** with project key **SCRUM** (Group 3 Jira space). The board has six epics – E1 Data Pipeline, E2 Model Development, E3 Edge Device (Raspberry Pi), E4 Login Integration, E5 MLOps & DevOps, E6 Project Management & Reporting – and five sprints (S0–S4) that match Section 1. User stories (Jira "Feature" work items) are estimated in story points; tasks carry an owner, a backup owner, a priority and hour estimates. Workflow: Idea → To Do → In Progress → Testing → Done (Testing is our review step). A task is *Done* when its acceptance criteria pass, the pull request is reviewed and merged, and any documentation is updated. Board link: **https://group3-cos40007.atlassian.net/jira/software/projects/SCRUM/boards/1**.

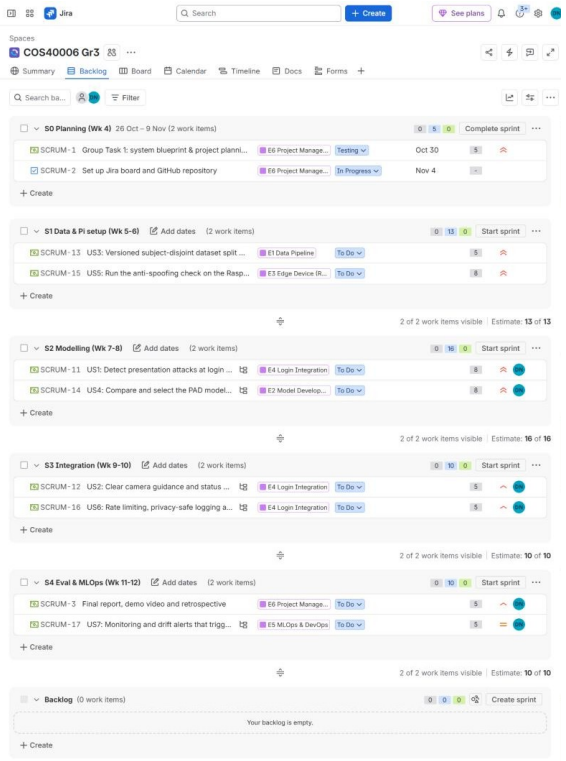


Figure 2. Jira backlog: sprints S0–S4 with the user stories (US1–US7), epics, owners, priorities and story-point estimates.

Interna
l

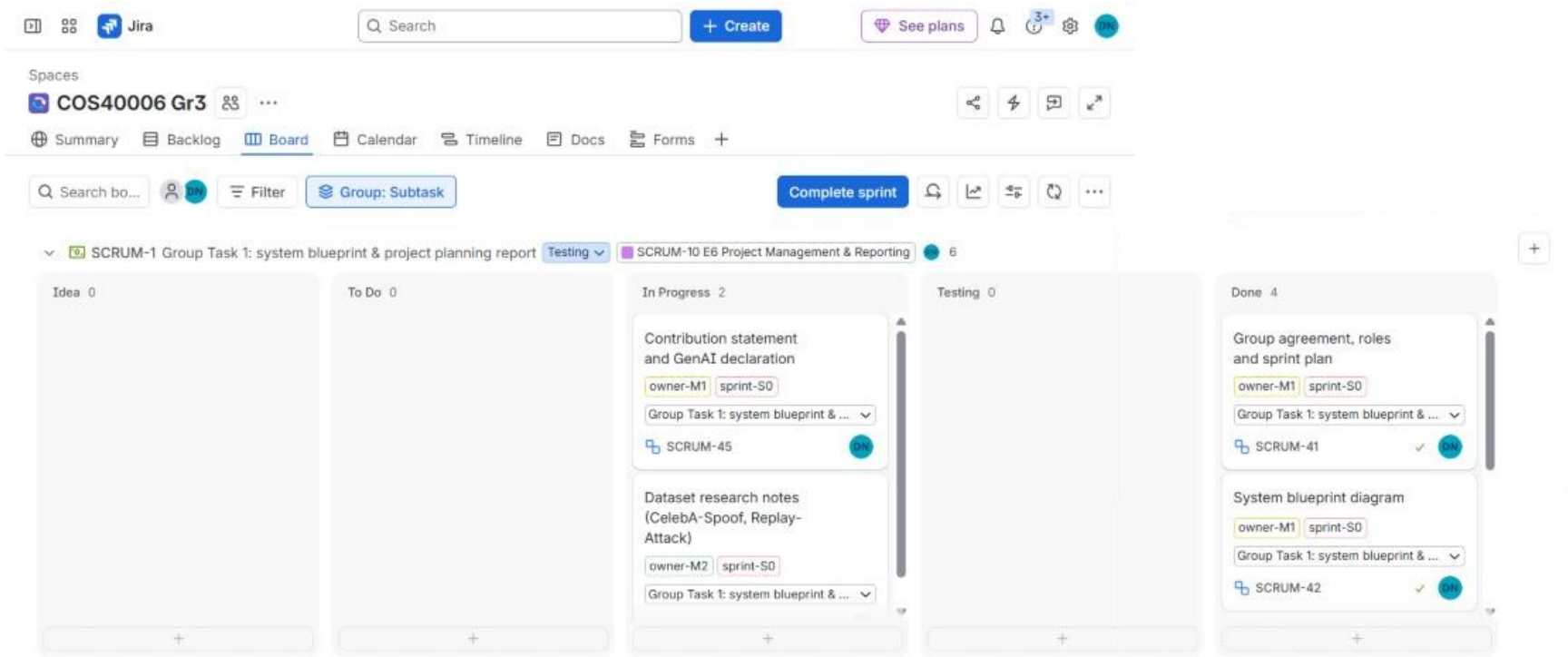


Figure 3. Jira board for the active sprint S0 (grouped by subtask): planning tasks moving through To Do, In Progress, Testing and Done.

4.2 User stories

ID	User story	Acceptance criteria	Priority	SP	Sprint	Owner / epic
US1	As a **user logging in**, I want the system to confirm that a live person is in front of the camera so that nobody can enter my account with my printed photo, a photo on a screen or a replayed video.	Print, screen and replay attacks are rejected in the demo; APCER, BPCER and ACER are reported per attack type on the held-out test set.	Highest	8	S2–S3	M1 / E4
US2	As a **user**, I want clear camera guidance and status messages so that I know how to pass the check and what to do when it fails.	Distinct messages for success, spoof suspected, low quality (dark, blurred, no face, several faces), timeout and technical failure; a retry button.	High	5	S3	M1 / E4

ID	User story	Acceptance criteria	Priority	SP	Sprint	Owner / epic
US3	As a **data engineer**, I want a versioned, subject-disjoint dataset split with a frozen test set so that model comparisons are fair and experiments are reproducible.	Manifest under DVC; automated check finds no subject overlap; test-set hash recorded before tuning.	Highest	5	S1	M1 / E1
US4	As an **ML engineer**, I want to compare candidate models on validation data so that we select the model with the lowest attack acceptance inside our latency budget.	At least three models logged in MLflow; threshold chosen on validation; comparison table with APCER/BPCER/ACER and Pi latency.	Highest	8	S2	M1 / E2
US5	As a **device operator**, I want the anti-spoofing check to run on the Raspberry Pi so that face images never leave the device and the check stays fast.	INT8 ONNX model runs on Pi 5; inference p95 ≤ 50 ms per frame and ≤ 2 s per attempt; no images stored; result signed.	Highest	8	S1–S3	M1 / E3
US6	As a **security administrator**, I want attempts to be rate-limited, logged without face images, and to fail closed so that attackers cannot brute-force or bypass the check and incidents can be audited.	Retry limit and lockout work; logs hold decision, score, model version, error code and timestamp only; a technical failure never results in a login.	High	5	S3	M1 / E4
US7	As an **ML engineer**, I want monitoring and drift alerts that trigger retraining so that the model stays reliable when lighting, cameras or attack types change.	Weekly drift report (PSI) and audit APCER/BPCER tracked; retraining rules documented and dry-run once.	Medium	5	S4	M1 / E5

4.3 Backlog and task breakdown

User stories are broken into tasks (imported into Jira as sub-tasks of each story, so Jira assigned SCRUM keys, e.g. T1 = SCRUM-18); each task has one owner and the other member is always the backup owner. M1 owns most tasks (≈ 134 h); M2 supports dataset research and data preparation (≈ 9 h). Priorities follow the MoSCoW scope in Section 1.

ID	Task	Story	Owner	Priority	Sprint	Est. (h)
T1	Download CelebA-Spoof; request Replay-Attack access via tutor; record licences	US3	M2	Highest	S1	4
T2	Build dataset manifest (path, label, subject, attack type, lighting, hash) and track with DVC	US3	M1	Highest	S1	6
T3	EDA: class balance, attack types, lighting, images per subject	US3	M2	High	S1	5
T4	Subject-disjoint 80/10/10 split, leakage and near-duplicate checks, freeze test set	US3	M1	Highest	S1	5
T5	Shared preprocessing module (YuNet detection, crop, resize, quality checks) + unit tests	US5	M1	Highest	S1	8
T6	Set up Pi 5 (OS, Camera Module 3, picamera2); capture module with camera/video/mock sources	US5	M1	Highest	S1	6
T7	Baselines: LBP+SVM and pre-trained MiniFASNet on validation	US4	M1	High	S2	6
T8	Fine-tune MiniFASNet and MobileNetV3-Small with augmentation; MLflow tracking	US4	M1	Highest	S2	12
T9	Evaluation script: APCER, BPCER, ACER, ROC-AUC, confusion matrix per attack type	US1	M1	Highest	S2	6
T10	Threshold selection on validation and model comparison table	US4	M1	High	S2	4

ID	Task	Story	Owner	Priority	Sprint	Est. (h)
T11	ONNX export, INT8 quantisation, FP32 vs INT8 parity check	US5	M1	High	S2	5
T12	Pi latency and resource benchmark (median/p95, temperature, throttling)	US5	M1	High	S2–S3	5
T13	Edge service: frame aggregation, decision engine, signed result (FastAPI + systemd)	US5	M1	Highest	S3	10
T14	Backend auth API: password check, nonce issue, signature verification	US6	M1	Highest	S3	8
T15	Web login UI with guidance and all status states	US2	M1	High	S3	10
T16	Error states: camera unavailable, consent declined, timeout	US2	M1	High	S3	4
T17	Rate limit, retry limit and lockout; structured logging without images	US6	M1	High	S3	5
T18	Automated tests: pass, spoof, low quality, camera denied, timeout, repeated failures	US6	M1	Highest	S3–S4	8
T19	End-to-end integration and demo script (genuine user + print/screen/replay attacks)	US1	M1	Highest	S3	6
T20	GitHub Actions CI: lint, pytest, data checks	US7	M1	Medium	S3	4
T21	One-time test-set evaluation and consented Pi-camera sanity test	US1	M1	High	S4	6
T22	Monitoring dashboard and weekly drift report (PSI)	US7	M1	Medium	S4	6
T23	Retraining trigger rules, release and rollback procedure – document and dry-run	US7	M1	Medium	S4	4

Project-management tasks (Task 1 report, final report, demo video, weekly minutes) sit under epic E6 and are shared by both members.

4.4 Project documentation (shared space)

Our shared documentation space is a Word page, *Project_Documentation.docx*, stored in Group Project/docs/ of the GitHub repository (https://github.com/phu1009/COS40007---AI-Engineering-). It contains the project description, the system overview (Figure 1 and Section 2.1), meeting minutes and the key design decisions below.

ID	Key design decision	Reason
DD1	Run the liveness check on the Raspberry Pi (edge), not in the cloud	Face images stay on the device; latency and resource use are measured on the real target hardware. Trade-off: limited compute, so only lightweight models.
DD2	Passive liveness first; blink challenge only as an optional second step	Passive detection covers print, screen and replay attacks with no user effort; a replayed video can contain blinks, so a blink check alone is weak.
DD3	Lightweight CNN (MiniFASNet / MobileNetV3-Small) exported to ONNX INT8; LBP+SVM as baseline	Under 5 MB, runs on the Pi CPU without an accelerator; the baseline shows what deep learning adds.
DD4	Subject-disjoint split, frozen test set, threshold chosen on validation	Prevents identity leakage and optimistic results, as required by the project specification.
DD5	Select the model by APCER at BPCER ≤ 5% plus Pi p95 latency, not by accuracy	Classes are imbalanced (about 1:3) and accepting an attack costs more than asking a genuine user to retry.

ID	Key design decision	Reason
DD6	Pi results are signed and bound to a server nonce; technical errors fail closed	Stops forged or replayed "pass" messages; a failure can never bypass the check.
DD7	No raw face images in logs; monitoring on aggregates; retraining only with consented data	Meets the privacy requirement of the specification and university ethics expectations.
DD8	GitHub + DVC + MLflow for versioning; GitHub Actions for CI/CD	Free tools that link code, data, experiments and models for full reproducibility.

5. Contribution Statement

Contributions below describe the work each member actually did on Group Task 1 and can be checked against the GitHub history.

Member	Contribution to Group Task 1	Share (%)	Agreed (initials)
Dinh Phu Nguyen **105027282**	• Analysed the task brief, rubric, template and project specification • Designed the system architecture and drew the system blueprint (Figure 1) • Wrote the group agreement, sprint plan and timeline (Section 1) • Wrote the MLOps & DevOps design (Section 3) • Wrote the user stories and task breakdown; set up the Jira board and the GitHub repository • Prepared the project documentation, compiled and proofread the report, and submitted it	90%	DPN
Phuc Lam Nguyen **105006540**	• Background research on candidate datasets (CelebA-Spoof, Replay-Attack) • Small data-preparation support	10%	

6. Generative AI Declaration

We used generative AI in this task. **Tool:** Claude (Anthropic), 21 and 25 October 2026. **Purpose:** (1) analysing the task brief, rubric and template; **Proof:** screenshots in Appendix A.

7. Links

- **Group GitHub repository:** https://github.com/phu1009/COS40007---AI-Engineering- – Task 1 files are in Group Project/docs/ (report in docs/task1/, blueprint source in docs/architecture/, Jira import in docs/jira/).
- **Jira board:** https://group3-cos40007.atlassian.net/jira/software/projects/SCRUM/boards/1
- **Project documentation:** Word file Group Project/docs/Project_Documentation.docx in the GitHub repository

References

Chingovska, I, Anjos, A & Marcel, S 2012, 'On the effectiveness of local binary patterns in face anti-spoofing', *Proceedings of the International Conference of the Biometrics Special Interest Group (BIOSIG)*, IEEE.

GitHub 2026, *GitHub Actions documentation*, GitHub Docs, viewed 25 October 2026, <https://docs.github.com/actions>.

ISO/IEC 30107-3:2023, *Information technology – Biometric presentation attack detection – Part 3: Testing and reporting*, International Organization for Standardization, Geneva.

Martinez-Diaz, Y et al. 2023, 'Exploring the effectiveness of lightweight architectures for face anti-spoofing', *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, IEEE.

Raspberry Pi Ltd 2026, *Camera software and AI Camera documentation*, viewed 25 October 2026, <https://www.raspberrypi.com/documentation/>.

Zhang, Y, Yin, Z, Li, Y, Yin, G, Yan, J, Shao, J & Liu, Z 2020, 'CelebA-Spoof: large-scale face anti-spoofing dataset with rich annotations', *European Conference on Computer Vision (ECCV)*, Springer, pp. 70–85.

Appendix A – GenAI evidence

Những điểm nhóm cần lưu ý

1. **Đặc tả không nhắc tới Raspberry Pi.** Đề chỉ ghi "web or mobile login prototype". Nên trình bày Pi là *target environment* cho phần inference, vì đề có nhắc tới "deployment constraints" và "resource use on the target environment". Nhóm nên hỏi tutor xác nhận kiến trúc này được chấp nhận.
2. **"Measured targets" chưa có số cụ thể.** Nhóm phải tự đặt mục tiêu, ví dụ p95 inference dưới 50 ms/khung, và chọn ngưỡng sao cho BPCER ≤ 5% rồi giảm APCER thấp nhất có thể.
3. **Dataset Replay-Attack:** người đăng ký phải là nhân viên chính thức của trường và không được dùng email cá nhân. Vì vậy cần nhờ giảng viên đăng ký ngay. CelebA-Spoof thì tự tải được, nhưng chỉ dùng phi thương mại.
4. **Domain gap:** Replay-Attack quay bằng webcam 320×240 từ năm 2012, rất khác camera Pi. Cần thêm một bộ test nhỏ tự quay bằng camera Pi, gồm thành viên nhóm (có đồng ý) và các kiểu tấn công bằng ảnh in, điện thoại, laptop.

Hướng làm phần nhúng trên Pi

Kiến trúc: Pi đóng vai trò "Liveness Edge Node". Luồng như sau:

1. Người dùng nhập mật khẩu trên web.
2. Backend cấp một nonce.
3. Pi chụp 10–15 khung hình, rồi lần lượt: phát hiện mặt → kiểm tra chất lượng → chạy model PAD → gộp điểm các khung → ra quyết định.
4. Pi **ký kết quả bằng khóa riêng** (gắn kèm nonce và timestamp) và gửi về backend qua HTTPS.